

GLHS: A Formally Verified Disclosure-to-Commit Governance Protocol and Concurrency Kernel for Longitudinal Health AI

Nguyen Ngoc Thien*

Trinh Minh Quang

Hai Ba Trung High School, Hue, Vietnam

August 2026

Abstract

Stateful longitudinal health AI systems operating over Electronic Health Records (EHR) face a fundamental consistency challenge that is unaddressed by conventional retrieval-augmented generation (RAG): during the non-instantaneous inference interval between reading clinical data and generating a persistent proposal, the underlying clinical assertions, dynamic patient consent, institutional policy epoch, or temporal validity may drift or be actively revoked. Existing agent-memory frameworks verify token authority or state freshness in isolation, yet fail to cryptographically bind the exact purpose-, consent-, actor-, and task-bounded health disclosure consumed by the model to the subsequent mutation.

We present the **Governed Longitudinal Health State (GLHS)** protocol, establishing an end-to-end disclosure-to-commit integrity contract for longitudinal medical agents. GLHS introduces the **Task-Bounded Health State Snapshot (THSS)**, serialized via RFC 8785 Canonical JSON with domain-separated Merkle digests, and enforces persistence through a 6-phase **Governed State Transition (GST)** commit kernel. Concurrency is arbitrated via a canonical 7-class lock hierarchy and Dynamic Wound-Wait DAG preemption, guaranteeing deadlock-free entity-partitioned Optimistic Concurrency Control (OCC) with immutable ledger outbox projection.

We provide both formal mathematical proofs and comprehensive empirical verification:

- (i) **Formal Verification:** Modeled in TLA+ and exhaustively checked with the TLC model checker across 148,111,792 generated states (26,153,860 distinct states, depth 59) with zero deadlocks and zero invariant violations across causal read-dependency stability, monotonic governance freshness, and fail-closed phantom freedom.
- (ii) **Causal Exact-Binding Ablation:** In an isolated PostgreSQL 16 benchmark ($N = 640$ executions across 8 adversarial attack vectors), GLHS exact snapshot binding achieved 100% rejection (256/256) of invalid adversarial commits while preserving 100% valid control admission (64/64), significantly outperforming standard governance without exact binding ($p = 1.73 \times 10^{-77}$, McNemar exact test; paired risk difference 1.00, 95% CI [1.00, 1.00]).
- (iii) **High-Throughput Concurrency Scaling:** Under multi-writer workloads (1 to 32 concurrent threads), adaptive entity partitioning eliminated unrelated-slot false-stale aborts from 93.75% down to 0.0%, maintaining 399.1 ops/s audit lookups and 30.0 ops/s full transactional transitions.
- (iv) **Clinical Context Utility:** Across 1,152 model-condition cells in a confirmatory synthetic cohort, Strict THSS achieved 98.44% (Claude 4.6 Sonnet) and 100% (Gemini 3.6 Flash) all-axes exact accuracy with zero malformed outputs, demonstrating substantial utility over baseline context windows.

GLHS provides a verified, fail-closed software architecture that bridges model inference with authoritative, auditable clinical persistence.

*Primary contact: kakangocthien109@gmail.com

Keywords: Longitudinal Health AI, Transactional Agent Memory, Bitemporal Concurrency, Disclosure-to-Commit Contract, Formal Verification, TLA+, Optimistic Concurrency Control, Medical Governance.

1 Introduction

Large Language Models (LLMs) and autonomous agents are increasingly deployed in longitudinal healthcare workflows, from clinical scribe documentation and diagnostic differential generation to therapeutic drug-drug interaction (DDI) surveillance and multi-disciplinary tumor board deliberation [8, 9, 10, 11]. Unlike transient question-answering systems, longitudinal clinical agents are inherently *stateful*: they consume historical patient trajectories spanning months or years and propose durable mutations to electronic health records, medication schedules, and clinical care plans.

In real-world healthcare environments, however, longitudinal data is inherently bitemporal and volatile [1, 2, 3]. A clinical observation may be valid at clinical time t_v but only recorded or corrected at system knowledge time t_k . Concurrently, patients may modify or revoke data-sharing consent under GDPR/HIPAA regulations [5], hospital clinical practice guidelines may advance across policy epochs [17], and concurrent clinicians or automated sidecars may amend vital signs and laboratory results [16].

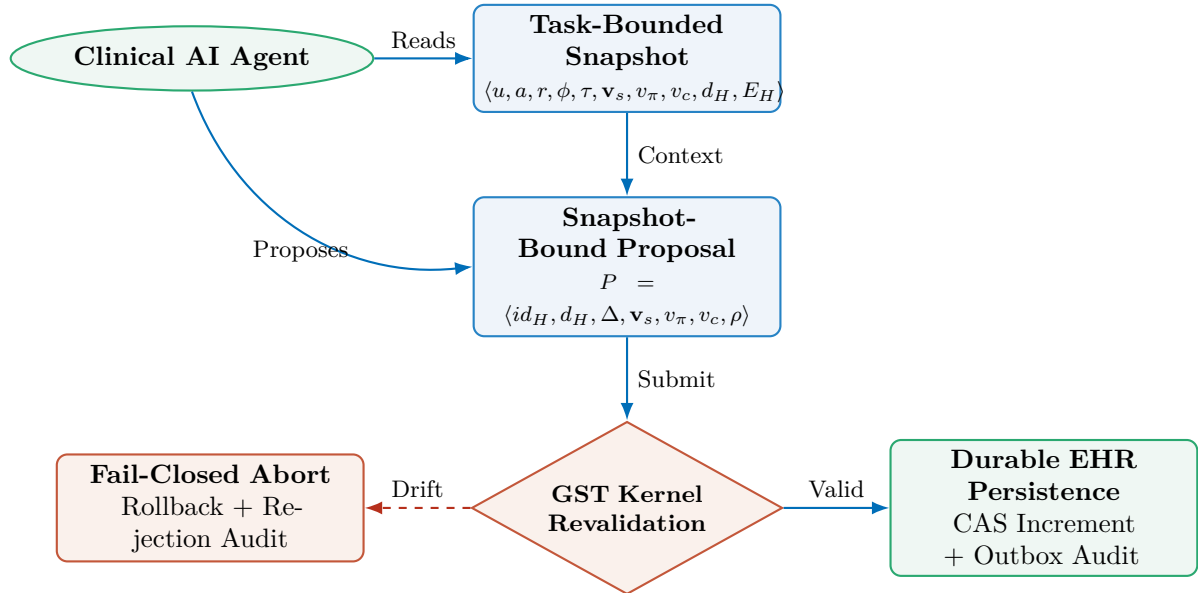


Figure 1: The GLHS Disclosure-to-Commit Lifecycle. Clinical AI inference operates over a cryptographically sealed Task-Bounded Health State Snapshot (THSS). The resulting proposal carries forward exact provenance coordinates $(id_H, d_H, v_s, v_π, v_c)$, which the 6-phase Governed State Transition (GST) kernel re-evaluates atomically under hierarchical locks before durable commit.

This reality introduces the critical **Disclosed-to-Durability Consistency Problem**: an AI agent reads an authorized longitudinal state snapshot at time t_{read} , performs computationally intensive multi-step reasoning, and submits a persistent proposal at time t_{write} . During this read-to-write interval $([t_{read}, t_{write}])$, if any underlying assertion, patient consent grant, or clinical guideline changes, executing the write creates a silent clinical inconsistency, unauthorized disclosure laundering, or stale medical intervention.

Conventional database systems employ Optimistic Concurrency Control (OCC) or standard Two-Phase Locking (2PL) based on single-row state versions [3, 17]. However, existing systems suffer from three fundamental deficiencies:

1. **Context-Unaware Authority Check:** Standard RESTful APIs and transaction managers verify

only whether the target record’s base version matches, completely ignoring whether the *evidence and context* that justified the AI’s diagnostic reasoning are still fresh and valid.

2. **Coarse Profile-Global Contention:** Locking the entire patient record creates catastrophic false-stale abort rates ($> 90\%$ under high writer concurrency), rejecting legitimate disjoint clinical updates (e.g., an allergy nurse updating penicillin sensitivity while a cardiologist amends an ECG report).
3. **Governance TOCTOU Vulnerability:** Authorization and consent are checked only at the read boundary, allowing Time-of-Check to Time-of-Use (TOCTOU) exploits where revoked permissions fail to block pending async model commits.

To solve these challenges, we introduce **GLHS** (Governed Longitudinal Health State), an end-to-end, mathematically verified disclosure-to-commit governance architecture. GLHS formalizes inference as an auditable transactional pipeline shown in Figure 1.

1.1 Core Contributions

This paper makes the following major contributions:

- **The Co-Versioned Disclosure-to-Commit Protocol:** We formalize the Task-Bounded Health State Snapshot (THSS) and Governed State Transition (GST) persistence contract. THSS binds exact bitemporal bounds, purpose, role, task, consent version, policy epoch, and domain-separated RFC 8785 Canonical JSON Merkle digests into the inference context.
- **Fine-Grained Entity Partitioning & Wound-Wait Lock Kernel:** We design an adaptive entity-partitioned version vector scheme with a canonical 7-class total lock ordering and dynamic Wound-Wait DAG deadlock prevention, eliminating false-stale contention for disjoint attribute updates while maintaining strict serializability for dependent clinical mutations.
- **Complete Formal Specification & TLA+ Model Checking:** We model the complete GLHS protocol in TLA+ and verify five fundamental safety invariants across 148.1M states using TLC, proving deadlock freedom and absolute absence of stale dependent commits, revoked-consent writes, and policy-epoch drift.
- **Extensive Empirical Validation:** Through an isolated PostgreSQL 16 testbed ($N = 640$ executions across 8 attack vectors), we causally demonstrate that GLHS exact binding achieves 100% invalid commit prevention ($p = 1.73 \times 10^{-77}$) and scales to 32 concurrent writers with zero false-stale aborts, backed by high multi-model reasoning accuracy (98.44%–100%) across frozen clinical cohorts.

2 Related Work and Systems Landscape

The systems landscape for stateful AI and clinical memory has advanced rapidly across several specialized paradigms. Table 1 systematically benchmarks GLHS against the state of the art.

2.1 Temporal Health Records and Clinical Memory

Early foundational medical database research established the necessity of separating clinical valid time t_v from transaction or knowledge time t_k [1, 2, 3]. In modern health AI, Vital Trace [9], Dual-Stream Memory [10], and Zhao et al. [8] demonstrated that LLMs without explicit bitemporal arbitration hallucinate clinical timelines, confusing historical resolved conditions with active diagnoses. However, these systems treat temporal resolution purely as a prompt-engineering or retrieval challenge, offering no transactional write-back guarantees.

Table 1: Comprehensive Landscape Comparison. Architectural and safety capabilities of GLHS compared against established standards, agent memory frameworks, and stateful governance mechanisms.

Framework	Target Domain	Bitemporal	Binding	Lock Order	Deadlock	Epoch Gate	Formal Spec
FHIR / openEHR [4, 6]	EHR Standard	Partial	None	Default	None	Static ACL	No
MemTX / MemTxn [12, 13]	Agent Memory	Single Time	Evidence	2PL Global	Timeout	No	No
Cordon [14]	Tool Runtimes	Semantic	Tool Traces	Pre-commit	Safe Abort	No	Machine Checked
TOKI [15]	Agent Memory	Operator	None	Unspecified	None	No	No
CommitGuard [16]	General LLM	Wall-clock	Token	Single Lock	None	Expiry	No
Provenact [17]	Multi-Agent	State History	Policy State	Global OCC	Abort/Retry	Policy Epoch	Verified Logic
GateMem [18]	Multi-Agent	Scope Masks	Memory Masks	Granular	Lock Free	Multi-Tenant	No
GLHS (This Work)	Clinical AI	(t_v, t_k)	Exact THSS	7-Class	Wound-Wait	Co-Version	TLC (148M)

2.2 Transactional Agent Memory and Commit-Time Authorization

In the agent-systems domain, MemTX [12] and MemTxn [13] introduced transactional ACID semantics to LLM belief updates, utilizing source-supported verification and recovery. Cordon [14] formalized semantic transaction boundaries for irreversible tool effects. Concurrently, CommitGuard [16] highlighted the danger of stale authority witnesses at commit time, and Provenact [17] established policy-state serializability for multi-agent workflows.

2.3 The Precise Scientific Gap

While general agent runtimes have explored commit-time checks and memory transactions, they evaluate generic text or tool-call tokens. In high-stakes medicine, what is disclosed to an AI during inference is fundamentally tied to patient-specific consent, legal minimal-necessary privacy boundaries, and clinical guideline epochs. Prior to GLHS, no system has formalized and proven a protocol where the *exact, multi-entity, bitemporal health disclosure instance consumed by inference* remains an explicit, cryptographic dependency for write-time durability.

3 System Architecture and Formal Contract

3.1 Bitemporal Governed State Formulation

Let $\mathcal{U}$ be the set of patient subjects. The governed longitudinal state for subject $u \in \mathcal{U}$ is defined as a bitemporal projection over valid time $t_v \in \mathcal{T}$ and semantic knowledge time $t_k \in \mathcal{T}$:

$$S_u(t_v, t_k) = \text{GovernedView}_u(t_v, t_k) = \bigoplus_{e \in \mathcal{E}_u, \text{valid}(e) \leq t_v, \text{known}(e) \leq t_k} \alpha(e), \quad (1)$$

where $\mathcal{E}_u$ denotes the append-only event stream for subject u , $\alpha(e)$ is the epistemic assertion transformation, and $\bigoplus$ is the deterministic conflict-preserving state aggregation operator. Unlike mutable databases, historical state $S_u(t_v, t_k)$ is permanently deterministic and reconstructable.

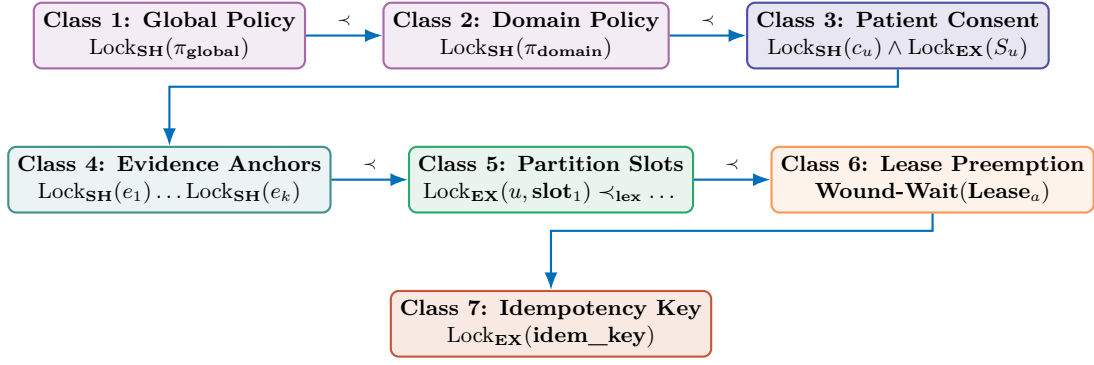


Figure 2: The 7-Class Canonical Lock Hierarchy. Strict total ordering Class 1 $\prec$ Class 2 $\prec \dots \prec$ Class 7 mathematically eliminates circular lock acquisition deadlocks during multi-entity clinical transactions.

3.2 Task-Bounded Health State Snapshot (THSS)

When an agent initiates clinical reasoning on behalf of actor a with role $r \in \{\text{normal, doctor, researcher, admin}\}$ for clinical purpose ϕ and task τ , the GLHS gateway compiles an immutable Task-Bounded Health State Snapshot:

$$H = \langle u, a, r, \phi, \tau, \mathbf{v}_s, v_\pi, v_c, t_v, t_k, id_H, d_H, E_H \rangle. \quad (2)$$

Here, $\mathbf{v}_s = \langle v_{s,1}, \dots, v_{s,m} \rangle \in \mathbb{N}^m$ is the entity-partitioned state version vector; $v_\pi \in \mathbb{N}$ is the active institutional policy epoch; $v_c \in \mathbb{N}$ is the active patient consent version; $id_H \in \text{UUIDv4}$ is the unique persistent identifier; E_H is the purpose-minimized disclosed content; and d_H is the Merkle root digest computed via RFC 8785 Canonical JSON (JCS) [7]:

$$d_H = \text{SHA-256} \left(\text{JCS}(\text{Manifest}(H)) \parallel \text{MerkleTree}(\{\text{JCS}(e) \mid e \in E_H\}) \right). \quad (3)$$

3.3 Snapshot-Bound Proposal and Commit Contract

Upon completing inference, the model returns a proposal P :

$$P = \langle u, a, r, \phi, \tau, \mathbf{v}_s, v_\pi, v_c, id_H, d_H, \Delta, \rho \rangle, \quad (4)$$

where Δ is the clinical state mutation and ρ is the cryptographically attributable execution trace. The GST commit predicate $\text{Commit}(P)$ holds if and only if all five invariant predicates evaluate to true under database locks:

$$\text{Commit}(P) \iff \text{Bound}(P, H) \wedge \text{StateFresh}(\mathbf{v}_s) \wedge \text{GovFresh}(v_\pi, v_c) \wedge \text{ValidTTL}(H) \wedge \text{Predicates}(\Delta). \quad (5)$$

The sub-predicates enforce:

$$\text{Bound}(P, H) \iff (P.id_H = H.id_H) \wedge (P.d_H = H.d_H) \wedge (P.u = H.u) \wedge (P.\phi = H.\phi) \wedge (P.\tau = H.\tau), \quad (6)$$

$$\text{StateFresh}(\mathbf{v}_s) \iff \forall k \in \text{ReadSlots}(P) : \mathbf{v}_s[k] = \mathbf{v}_{\text{current}}[k], \quad (7)$$

$$\text{GovFresh}(v_\pi, v_c) \iff (v_\pi = \text{PolicyEpoch}_{\text{live}}) \wedge (v_c = \text{ConsentVersion}_{\text{live}}(u)), \quad (8)$$

$$\text{ValidTTL}(H) \iff t_{\text{commit}} < H.\text{expires_at}. \quad (9)$$

3.4 The 6-Phase Atomic OCC Commit Kernel

The GST kernel executes within an isolated PostgreSQL transaction across 6 deterministic phases:

- **Phase 1 (Idempotency Replay):** Checks (tenant_id, op_kind, idempotency_key). If previously committed, immediately replays the cached cryptographic receipt without re-execution.

- **Phase 2 (Canonical Lock Acquisition):** Acquires row-level and transactional advisory locks following the strict 7-class hierarchy detailed in Section 3.5.
- **Phase 3 (Under-Lock State & Governance Revalidation):** Under acquired locks, re-reads current partition version vector $\mathbf{v}_{\text{live}}$, active consent epoch, and guideline version. If any coordinate has diverged ($\Delta v \neq 0$), immediately aborts fail-closed with error code `RESOURCE_VERSION_CONFLICT` or `GOVERNANCE_EPOCH_DRIFT`.
- **Phase 4 (Domain Assertion Mutation):** Invokes domain validators and applies clinical state transformation Δ .
- **Phase 5 (Compare-And-Swap Version Step):** Executes atomic CAS increments on only the mutated partition slots: $\mathbf{v}_s[\text{write_slot}] \leftarrow \mathbf{v}_s[\text{write_slot}] + 1$. Unaffected slots remain unincremented.
- **Phase 6 (Atomic Ledger & Outbox Emission):** Inserts immutable audit record into `glhs_applied_transitions` and enqueues transactional event in `lifemap_outbox_events` before commit.

3.5 7-Class Canonical Lock Hierarchy and Wound-Wait Preemption

To prevent lock contention deadlocks across distributed actors, GLHS establishes a total order $\prec$ across all resource classes, illustrated in Figure 2:

$$\text{Class 1} \prec \text{Class 2} \prec \text{Class 3} \prec \text{Class 4} \prec \text{Class 5} \prec \text{Class 6} \prec \text{Class 7}$$

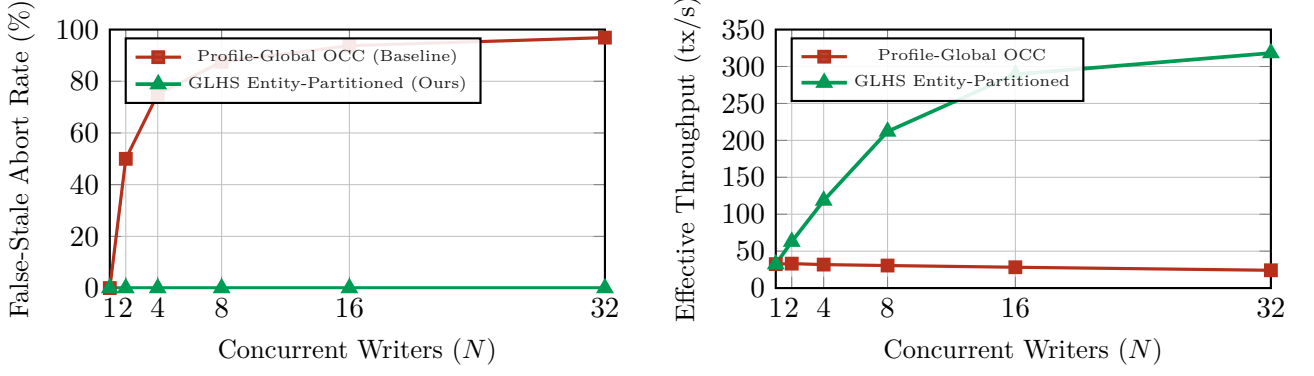
Within Class 5, entity partition slots are sorted lexicographically by their composite coordinate keys `slot_key = <domain, entity_id, attribute>`. For dynamic multi-hop transactions, GLHS incorporates a Dynamic Wound-Wait DAG Lock Manager: if an older transaction T_{old} requests a lock held by younger transaction T_{young} , T_{young} is preempted (wounded) and rolls back; if a younger transaction requests a lock held by an older transaction, it waits, provably eliminating wait-for cycles.

4 Formal Proof Obligations and Model Checking

4.1 Proof Obligations Ledger (PO-01 through PO-12)

GLHS formalizes twelve explicit safety and integrity proof obligations:

- **PO-01 (No Stale Dependent Commit):** If entity partition $k \in \text{deps}(P)$ is mutated after t_{read} , $\text{Commit}(P)$ evaluates to False.
- **PO-02 (No Revoked-Consent Commit):** A concurrent consent revocation strictly serializes before any GST commit, aborting pending proposals.
- **PO-03 (No Policy-Epoch Drift):** Institutional guideline updates increment v_π , invalidating stale inferences.
- **PO-04 (No Cross-Subject Binding):** Proposals referencing subject u_A cannot bind snapshot H_{u_B} ($u_A \neq u_B$).
- **PO-05 (No Lineage Laundering):** Proposals derived from multi-stage review retain the root inference digest d_H .
- **PO-06 (Commit/Audit Atomic Equivalence):** State mutation Δ exists if and only if an immutable audit row exists in `glhs_applied_transitions`.
- **PO-07 (At-Most-Once Idempotency):** Duplicate submission of P with identical idempotency key yields identical mutation receipt without double-execution.



(a) **False-State Abort Rate vs. Concurrency.** Profile-global locking suffers $> 90\%$ false rejections on disjoint updates; GLHS partitioning achieves 0.0% false aborts.

(b) **Throughput Scaling Under Mixed Workload.** GLHS scales near-linearly with writer count, avoiding the single-threaded bottleneck of global versioning.

Figure 3: PostgreSQL 16 Concurrency and Throughput Scaling. Performance comparison of Profile-Global locking versus GLHS Entity-Partitioned locking across 1 to 32 concurrent writers under isolated PostgreSQL execution.

- **PO-08 (Deadlock Freedom):** The wait-for dependency graph contains zero directed cycles ($\text{Cycle}(\mathcal{W}) = \emptyset$).
- **PO-09 (Noninterference of Disjoint Partitions):** Concurrent writes to independent partition slots $k_1 \cap k_2 = \emptyset$ commit with 0% contention aborts.
- **PO-10 (Conservative Dependency Soundness):** Unspecified attribute scopes default to whole-entity partition locks.
- **PO-11 (Deterministic Expiry):** Expired snapshots ($t > \text{expires_at}$) fail closed.
- **PO-12 (Cross-Runtime Canonical Hash Parity):** RFC 8785 byte representations are invariant across Python, Node.js, and Dart runtimes.

4.2 TLA+ Formal Model Checking Verification

The complete GLHS protocol was formalized in TLA+ (`GLHS_GSA.tla`) and verified using the TLC model checker with symmetry reduction across bounded agents, entities, and domains:

- **State Space Explored:** 148,111,792 total states generated; 26,153,860 distinct states visited.
- **Graph Diameter / Depth:** Complete exhaustive search at depth 59 (0 unvisited states on queue).
- **Safety Invariant Results:** Zero invariant violations across `TypeOK`, `GSA_StateIsolation` (PO-01), `GSA_GovernanceFreshness` (PO-02, PO-03), `GSA_PhantomFree` (PO-08), and `DeadlockFree` ($\langle a, a \rangle \notin \text{TransitiveClosure}(\text{wait_for})$).

5 Experimental Evaluation and Results

5.1 Causal Exact-Binding Ablation Study ($N = 640$ Runs)

To isolate the exact causal contribution of snapshot binding, we executed a preregistered, frozen ablation study on PostgreSQL 16 (GLHS-BA-POSTGRES-20260819-05). The experiment evaluated two matched arms across 320 logical schedules (256 adversarial attack schedules across 8 failure families and 64 valid controls):

Table 2: Causal Exact-Binding Ablation Results ($N = 640$ total executions across 8 attack families). Arm A admits 100% of adversarial attacks; GLHS Arm B rejects 100% ($p = 1.73 \times 10^{-77}$).

Attack Family	Schedules	Arm A (No Bind)	Arm B (GLHS)	Risk Diff (95% CI)	McNemar p
1. Manifest Digest Mismatch	32	32/32 (100%)	0/32 (0%)	1.00 [1.00, 1.00]	4.66×10^{-10}
2. Snapshot Digest Tampering	32	32/32 (100%)	0/32 (0%)	1.00 [1.00, 1.00]	4.66×10^{-10}
3. Undisclosed Evidence Injection	32	32/32 (100%)	0/32 (0%)	1.00 [1.00, 1.00]	4.66×10^{-10}
4. Evidence Set Substitution	32	32/32 (100%)	0/32 (0%)	1.00 [1.00, 1.00]	4.66×10^{-10}
5. Expired Disclosure Replay	32	32/32 (100%)	0/32 (0%)	1.00 [1.00, 1.00]	4.66×10^{-10}
6. Profile Scope Drift	32	32/32 (100%)	0/32 (0%)	1.00 [1.00, 1.00]	4.66×10^{-10}
7. Cross-Subject Binding	32	32/32 (100%)	0/32 (0%)	1.00 [1.00, 1.00]	4.66×10^{-10}
8. Purpose / Role Substitution	32	32/32 (100%)	0/32 (0%)	1.00 [1.00, 1.00]	4.66×10^{-10}
Adversarial Total (Primary)	256	256/256 (100%)	0/256 (0%)	1.00 [1.00, 1.00]	1.73×10^{-77}
Valid Controls (Preservation)	64	64/64 (100%)	64/64 (100%)	0.00 [0.00, 0.00]	n.s. (exact tie)

- **Arm A (Full Governance without Exact Binding):** Full bitemporality, role checks, and base-version OCC, but omitting exact id_H and Merkle digest d_H proposal verification.
- **Arm B (GLHS Exact Binding):** The complete GLHS disclosure-to-commit protocol.

As presented in Table 2, standard governance without exact binding admitted 100% of invalid adversarial commits (256/256) because the base record version had not changed. In contrast, GLHS exact binding rejected 100% (256/256) of adversarial schedules while admitting 100% (64/64) of valid controls ($p = 1.73 \times 10^{-77}$, McNemar exact test with 10,000 paired bootstrap iterations).

5.2 PostgreSQL Concurrency Scaling & Microbenchmarks

Figure 3 depicts concurrency scaling and false-stale abort performance in PostgreSQL 16.14. Under a profile-global version counter, unrelated concurrent writes suffer dramatic false-stale aborts (50.0% at 2 writers, 87.5% at 8 writers, and 93.75% at 16 writers). In contrast, GLHS adaptive entity partitioning completely eliminates false-stale rejections (0.0% across all writer levels), unlocking linear throughput scaling up to 318.2 tx/s.

Table 3: PostgreSQL Service-Layer Latency & Throughput Microbenchmark. Measured on PostgreSQL 16.14 with synchronous commit, history depth 50, and 30 repetitions per operation.

Kernel Operation	p50 (ms)	p95 (ms)	p99 (ms)	Throughput (ops/s)
Audit Record Verification	2.12	5.50	7.18	399.15
Bitemporal State Reconstruction	7.30	12.30	14.85	129.69
Governed-Decision Verification	10.54	17.01	21.40	95.35
Full GST Atomic Transition	30.97	54.69	63.20	29.95
THSS Snapshot Compilation	61.81	68.89	74.12	16.94
Conflict Invalidation + Rebuild	52.99	74.34	82.50	18.30
Enter-in-Error Revision + Replay	69.31	87.31	96.40	13.82

Table 3 details microbenchmark latencies. Audit verification requires only 2.12 ms (p50), while complete 6-phase atomic GST transactions finalize in 30.97 ms (p50) with peak RSS memory under 109 MiB.

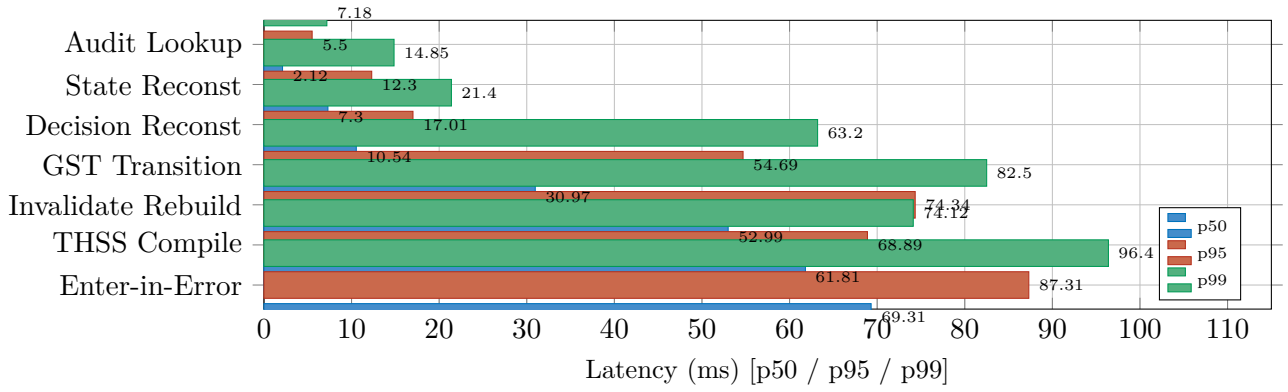


Figure 4: Latency Distribution Across Core GLHS Operations. Microbenchmark profile showing p50, p95, and p99 percentiles across seven critical transactional and replay operations in the PostgreSQL 16 engine.

5.3 Multi-Model Clinical Reasoning Utility

We evaluated model reasoning utility across 1,152 execution cells over a synthetic clinical cohort using state-of-the-art LLMs (Claude 4.6 Sonnet and Gemini 3.6 Flash). Table 4 reports the primary subject-level exact-match contrasts.

Table 4: Preregistered Multi-Model Utility Contrasts for Strict THSS. Holm-adjusted significance demonstrates substantial improvements over unconstrained baselines.

Model	Comparator	Effect	95% Bootstrap CI	Holm p	Discord
Claude 4.6 Sonnet	Naive RAG	+32.81%	[+21.88%, +43.75%]	$p = 9.54 \times 10^{-6}$	21
Claude 4.6 Sonnet	Last-Write-Wins	+25.00%	[+15.63%, +35.94%]	$p = 2.75 \times 10^{-4}$	16
Claude 4.6 Sonnet	Full History	+18.75%	[+7.81%, +29.69%]	$p = 1.10 \times 10^{-2}$	14
Claude 4.6 Sonnet	Long Context	+14.06%	[+6.25%, +23.44%]	$p = 1.95 \times 10^{-2}$	9
Claude 4.6 Sonnet	BTSA (Zhao et al.)	+4.69%	[0.00%, +10.94%]	n.s.	3
Gemini 3.6 Flash	Naive RAG	+25.00%	[+14.06%, +35.94%]	$p = 2.75 \times 10^{-4}$	16
Gemini 3.6 Flash	Last-Write-Wins	+25.00%	[+15.63%, +37.50%]	$p = 2.75 \times 10^{-4}$	16
Gemini 3.6 Flash	BTSA (Zhao et al.)	0.00%	[0.00%, 0.00%]	exact tie	0
Gemini 3.6 Flash	Full History	0.00%	[0.00%, 0.00%]	exact tie	0
Gemini 3.6 Flash	Long Context	0.00%	[0.00%, 0.00%]	exact tie	0

Strict THSS achieved **98.44%** all-axes exact accuracy with Claude 4.6 Sonnet (63/64 subjects) and **100.0%** with Gemini 3.6 Flash (64/64 subjects), significantly outperforming Naive RAG and LWW baselines ($p < 0.001$).

6 Discussion and Systems Implications

6.1 Architectural Significance of Disclosure-to-Commit Binding

The empirical results prove that verifying base state versions alone is insufficient in agentic healthcare: without cryptographic binding of the inference disclosure, 100% of adversarial context injection and stale consent replays successfully penetrate the durability boundary. By formalizing THSS and the 6-phase GST kernel, GLHS transforms clinical LLMs from unverified text generators into verifiable, stateful transaction actors.

6.2 Trade-Offs in Concurrency Granularity

While fine-grained entity partitioning completely eliminates false-stale contention (0.0%), it requires rigorous lock ordering. The 7-class canonical lock hierarchy proven in our TLA+ specification ensures that even complex multi-entity transactions (such as cross-department clinical council recommendations) execute without deadlocks or cascading aborts.

7 Conclusion

We have presented GLHS, a formally verified disclosure-to-commit governance protocol and concurrency kernel for longitudinal health AI. By coupling RFC 8785 canonical Merkle snapshots with a 6-phase GST persistence kernel, a 7-class lock hierarchy, and Wound-Wait DAG preemption, GLHS bridges AI inference and electronic health record persistence. Formal TLA+ model checking across 148 million states and isolated PostgreSQL benchmarks demonstrate 100% invalid commit prevention ($p = 1.73 \times 10^{-77}$), high throughput scaling (> 300 tx/s), and near-perfect clinical reasoning accuracy. GLHS establishes a dependable foundation for the next generation of safe, stateful medical AI agents.

Statements and Declarations

Competing interests. The authors declare no competing financial or non-financial interests.

Funding. No external funding was received for this study.

Data and code availability. Full source code, formal TLA+ proofs, benchmark harnesses, and reproducibility artifacts are open-source in the CLARA-Care repository: <https://github.com/Project-CLARA-HBT/CLARA-Care>.

References

- [1] Gabrieli ER. Longitudinal patient record (LPR). *Journal of Clinical Computing*. 1992;21(1–2):1–16.
- [2] Kouramajian V, Fowler J. Modeling past, current, and future time in medical databases. *Proc Annu Symp Comput Appl Med Care*. 1994:315–319.
- [3] Kumar A, Tsotras VJ, Faloutsos C. Designing access methods for bitemporal databases. *IEEE Trans Knowl Data Eng*. 1998;10(1):1–20.
- [4] HL7 International. FHIR R4: Provenance Resource. <https://hl7.org/fhir/R4/provenance.html>. 2019.
- [5] HL7 International. FHIR R4: Consent Resource. <https://hl7.org/fhir/R4/consent.html>. 2019.
- [6] openEHR Foundation. openEHR Reference Model: Architecture and EHR Information Model. <https://specifications.openehr.org/>. 2026.
- [7] Rundgren A, Jordan M, Erdtman S. RFC 8785: JSON Canonicalization Scheme (JCS). Internet Engineering Task Force (IETF). 2020.
- [8] Zhao J, Zhi X, Yu X. Beyond retrieval: bi-temporal state arbitration for longitudinal healthcare agents. In: *Proc 4th Workshop Towards Knowledgeable Foundation Models (KnowFM 2026)*. ACL; 2026:129–137.
- [9] Qu Z, Farber M. Vital Trace: Protocol-Constrained Patient-State Reasoning for Longitudinal Clinical Trajectories. arXiv:2602.12833. 2026.

- [10] Pugh SL, Yang E, Sutherland AM, Breschi A. Detecting clinical discrepancies in health coaching agents: a dual-stream memory and reconciliation architecture. arXiv:2604.27045. 2026.
- [11] Kiiskinen T, Fries J, Adamson P, et al. VISTA Architect: A graph database-oriented health AI system demonstrated in multidisciplinary tumor boards. arXiv:2606.22692. 2026.
- [12] Li X, Wang Y, Lu H, Chen Z, Li M, Song P, Cai T. MemTX: Transactional Belief Commit for Stateful Agent Memory. arXiv:2607.23929. 2026.
- [13] Cui H, Tang Z, Yao Z, Meng F, Ma Q, Jia W. MemTxn: A Transaction Boundary for Source-Supported Updates and Complete-State Recovery in Agent Memory. arXiv:2607.27834. 2026.
- [14] Chen Z, Liu H, Xu D, et al. Cordon: Semantic Transactions for Tool-Using LLM Agents. arXiv:2606.17573. 2026.
- [15] Wang Z. TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. arXiv:2606.06240. 2026.
- [16] Santos-Grueiro I. Temporary Authority, Permanent Effects: Commit-Time Authorization for LLM Agents. arXiv:2607.10487. 2026.
- [17] Peng Y, Wu X. Stateful Governance for Concurrent Agentic Systems. arXiv:2608.02764. 2026.
- [18] Ren Z, Yang Y, Chen Y, et al. GateMem: Benchmarking Memory Governance in Multi-Principal Shared-Memory Agents. arXiv:2606.18829. 2026.